

6. Technical Block Diagram & System Architecture

6.1 System Architecture Overview

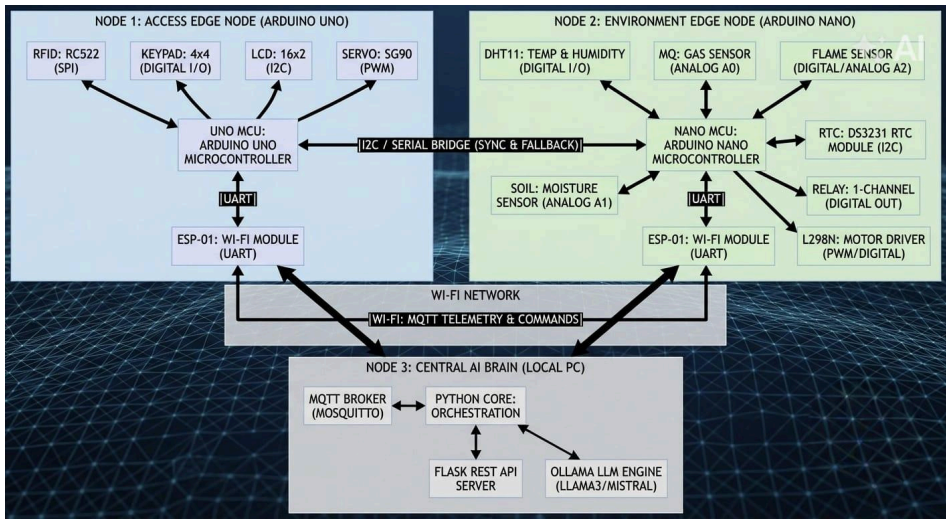
The AI-Integrated Smart Home Automation System utilizes a distributed three-node architecture designed to decouple real-time hardware sensing and actuation from high-level reasoning and decision-making.

- **Node 1 (Access Edge Node)** and **Node 2 (Environment Edge Node)** function as low-latency hardware execution units. They handle direct sensor acquisition, peripheral state management, and primary actuation.
- **Node 3 (Central AI Brain)** operates on a dedicated local PC host, running a local Python backend, an MQTT broker, a Flask REST server, and an Ollama LLM inference engine for localized intelligence without cloud dependency.

Data routing utilizes a hybrid topology:

- High-frequency telemetry and action commands flow wirelessly between the edge microcontrollers (via ESP-01 Wi-Fi modules) and Node 3 using MQTT pub/sub and HTTP REST payloads.
- Local critical sync and emergency fallback routines between Node 1 and Node 2 are maintained through a wired I2C / Serial Inter-Node Bridge, ensuring basic home functions persist even during Wi-Fi or central server offline events.

6.2 System Architecture Diagram



6.3 Hardware Pin & Protocol Mapping

Node	Peripheral / Sensor	Interface / Protocol	Operational Function
Node 1 (Uno) ▾	RFID RC522 ▾	SPI (MISO, MOSI, SCK, SS)	Contactless access card token authentication
Node 1 (Uno) ▾	4×4 Matrix Keypad ▾	Digital I/O (8 Pin Matrix)	PIN entry input for security verification
Node 1 (Uno) ▾	16×2 LCD Display ▾	I2C (SDA, SCL - Address 0x27)	Real-time status prompts and user feedback
Node 1 (Uno) ▾	SG90 Micro Servo ▾	PWM (Digital Pin 9)	Physical door lock mechanism actuation
Node 1 (Uno) ▾	ESP-01 Wi-Fi Module ▾	UART (SoftwareSerial Pin 10/11)	Wireless MQTT message transport to central server

Node	Peripheral / Sensor	Interface / Protocol	Operational Function
Node 2 (Nano) ▾	DHT11 Sensor ▾	Digital Single-Bus (Pin D4)	Ambient temperature and relative humidity sensing
Node 2 (Nano) ▾	MQ Gas Sensor ▾	Analog Input (Pin A0)	Combustible gas and air quality monitoring
Node 2 (Nano) ▾	Flame Sensor ▾	Digital / Analog (Pin A2 / D2)	Immediate optical fire and IR radiation detection
Node 2 (Nano) ▾	Soil Moisture Sensor ▾	Analog Input (Pin A1)	Automated plant hydration measurement
Node 2 (Nano) ▾	DS3231 RTC Module ▾	I2C (SDA, SCL - Address 0x68)	Precise hardware timestamping for offline events
Node 2 (Nano) ▾	1-Channel Relay ▾	Digital Output (Pin D7)	High-voltage load control (e.g., lighting/HVAC)
Node 2 (Nano) ▾	L298N Motor Driver ▾	PWM / Digital (Pins D5, D6, D8, D9)	Bi-directional DC motor control for ventilation
Node 2 (Nano) ▾	ESP-01 Wi-Fi Module ▾	UART (Hardware Serial RX/TX)	Environmental telemetry transmit and command listener

6.4 Data Flow Protocols & Telemetry Lifecycle

6.4.1 Inter-Node Local Communication (I2C / Serial Bridge)

Node 1 (Arduino Uno) acts as the Primary Master, while Node 2 (Arduino Nano) operates as a Secondary Slave on the local I2C bus (address 0x08).

- **Heartbeat & Sync:** Node 1 queries Node 2 every 1000ms for system health metrics and emergency flags (e.g., gas leak or fire detected).

- **Fallback Mode:** If connection to Node 3 (Central AI) drops, Node 1 directly commands Node 2 via this local bus to trigger localized routines (such as opening ventilation fans during gas alerts or pulsing local relays).

6.4.2 Wireless Telemetry Payload Formats

Telemetry payloads are transmitted asynchronously over MQTT as lightweight JSON strings.

Node 1 Access Event Payload (`home/nodes/access/event`):{

```
"node_id": "NODE_01_ACCESS",  
  
"timestamp": "2026-09-23T20:35:38Z",  
  
"event": "ACCESS_ATTEMPT",  
  
"auth_type": "RFID",  
  
"card_uid": "A3-89-B2-F0",  
  
"status": "VERIFYING"  
}
```

Node 2 Environmental Telemetry Payload (`home/nodes/env/telemetry`):{

```
"node_id": "NODE_02_ENV",  
  
"timestamp": "2026-09-23T20:35:38Z",  
  
"sensors": {  
  
  "temperature_c": 24.5,  
  
  "humidity_pct": 58.0,  
  
  "gas_ppm": 120,  
  
  "flame_detected": false,
```

```

    "soil_moisture_pct": 42.5

},

"actuators": {

    "relay_active": false,

    "fan_speed": 0

}

}

```

6.4.3 Central AI Decision Lifecycle & Actuation Dispatch

1. **Ingestion & Aggregation:** The Python core daemon ingests telemetry payloads from MQTT brokers and persists recent windowed metrics into local storage.
2. **Inference Trigger:** When anomalous sensor events occur or natural language user commands arrive via Flask REST API, Python structures a prompt context and queries the Ollama LLM Engine locally.
3. **Actuation Command:** The LLM outputs structured action directives, which the Python service formats and dispatches over MQTT to specific target edge topics.

Node 3 Actuation Command Payload (**home/nodes/access/cmd**):n

```

{
  "command_id": "CMD_88294",
  "target_node": "NODE_01_ACCESS",
  "action": "UNLOCK_SERVO",
  "duration_sec": 5,
  "reason": "AI_SCHEDULE_MATCH_AUTHORIZED"
}

```
